Tugas Pertemuan 2	
IF21W0508 – Praktikum Web	
NPM	243040091
Nama	Adriel Reihansyah Setiawan
Kelas	E
Link Github Repository	https://github.com/AdrielReihansyah/PW2026_243040091

Bagian 1. Screenshot Semua Kode

App.jsx



```jsx
// src/App.jsx
import { useState, useEffect } from 'react';
import './App.css';
import Kartu from './Kartu';
import Penghitung from './Penghitung';
import FormTugas from './FormTugas';
import ItemTugas from './ItemTugas';

function App() {
  // 1. State utama: ambil dari localStorage, jika kosong pakai data contoh
  const [daftarTugas, setDaftarTugas] = useState(() => {
    const dataTersimpan = localStorage.getItem('daftar-tugas');
    if (dataTersimpan) {
      return JSON.parse(dataTersimpan); // teks JSON -> array
    }
    return [
      { id: 1, judul: 'Install Node.js dan Vite', selesai: true },
      { id: 2, judul: 'Pelajari useState', selesai: false },
    ];
  });

  // State untuk Tugas 1: pilihan filter ('semua' | 'belum' | 'selesai')
  const [filter, setFilter] = useState('semua');

  // Efek A: dijalankan SEKALI saat App pertama kali tampil
  useEffect(() => {
    console.log('Aplikasi Daftar Tugas siap digunakan!');
  }, []);

  // Efek B: ubah judul tab setiap daftarTugas berubah
  useEffect(() => {
    const sisa = daftarTugas.filter((tugas) => !tugas.selesai).length;
    document.title = `(${sisa}) Tugas Belum Selesai`;
  }, [daftarTugas]);

  // Efek C: simpan ke localStorage setiap daftarTugas berubah
  useEffect(() => {
    localStorage.setItem('daftar-tugas', JSON.stringify(daftarTugas));
  }, [daftarTugas]);

  // 2. Tambah tugas: array BARU berisi tugas lama + tugas baru
  const tambahTugas = (judul) => {
    const tugasBaru = { id: Date.now(), judul: judul, selesai: false };
    setDaftarTugas([...daftarTugas, tugasBaru]);
  };
```



```jsx
  // 3. Ubah status: salin tugas yang dicentang, biarkan yang lain
  const ubahStatus = (id) => {
    const daftarBaru = daftarTugas.map((tugas) => {
      if (tugas.id === id) {
        return { ...tugas, selesai: !tugas.selesai };
      }
      return tugas;
    });
    setDaftarTugas(daftarBaru);
  };

  // 4. Hapus tugas: ambil semua KECUALI yang id-nya sama
  const hapusTugas = (id) => {
    setDaftarTugas(daftarTugas.filter((tugas) => tugas.id !== id));
  };

  // Tugas 2: hapus semua tugas yang sudah selesai
  const hapusSelesai = () => {
    setDaftarTugas(daftarTugas.filter((tugas) => !tugas.selesai));
  };

  // 5. Nilai turunan (dihitung dari state, bukan state baru)
  const jumlahSelesai = daftarTugas.filter((tugas) => tugas.selesai).length;

  // Nilai turunan untuk progress bar (persen tugas selesai)
  const persen = daftarTugas.length === 0
    ? 0
    : Math.round((jumlahSelesai / daftarTugas.length) * 100);

  // Tugas 1: saring daftar sesuai filter SEBELUM di-map
  const tugasTampil = daftarTugas.filter((tugas) => {
    if (filter === 'selesai') return tugas.selesai;
    if (filter === 'belum') return !tugas.selesai;
    return true; // 'semua'
  });

  // Fungsi kecil: tombol aktif = biru, tidak aktif = abu-abu
  const kelasFilter = (nama) =>
    filter === nama ? 'tombol tombol-kecil' : 'tombol tombol-kecil tombol-abu';

  return (
    <div className="wadah">
      <header className="hero">
        <h1>Daftar Tugas Praktikum</h1>
        <p>Laboratorium Informatika - Modul 2: React Lanjutan</p>
        <div className="identitas">
          <span>@ Adrial Raihansyah Satiawan</span>
```



```jsx
  return (
    <div className="wadah">
      <header className="hero">
        <h1>Daftar Tugas Praktikum</h1>
        <p>Laboratorium Informatika - Modul 2: React Lanjutan</p>
        <div className="identitas">
          <span>👤 Adriel Reihansyah Setiawan</span>
          <span>🪪 NPM: 243040091</span>
        </div>
      </header>

      <Penghitung />

      <Kartu judul="Tambah Tugas Baru">
        <FormTugas onTambah={tambahTugas} />
      </Kartu>

      <Kartu judul="Daftar Tugas">
        <p className="ringkasan">
          {jumlahSelesai} dari {daftarTugas.length} tugas selesai ({persen}%)
        </p>

        {/* Progress bar: lebarnya mengikuti persen */}
        <div className="progres">
          <div className="isi-progres" style={{ width: persen + '%' }}></div>
        </div>

        {/* Tugas 1: tombol filter dengan className dinamis */}
        <div className="baris filter-baris">
          <button className={kelasFilter('semua')} onClick={() => setFilter('semua')}>Semua</button>
          <button className={kelasFilter('belum')} onClick={() => setFilter('belum')}>Belum Selesai</button>
          <button className={kelasFilter('selesai')} onClick={() => setFilter('selesai')}>Selesai</button>
        </div>
```



```jsx
        {/* Ternary: pesan jika tidak ada tugas yang ditampilkan */}
        {tugasTampil.length === 0 ? (
          <p className="kosong">
            {daftarTugas.length === 0
              ? 'Belum ada tugas. Yuk tambahkan tugas pertamamu!'
              : 'Tidak ada tugas pada filter ini.'}
          </p>
        ) : (
          <ul className="daftar">
            {tugasTampil.map((tugas) => (
              <ItemTugas
                key={tugas.id}
                tugas={tugas}
                onUbahStatus={ubahStatus}
                onHapus={hapusTugas}
              />
            ))}
          </ul>
        )}

        {/* Tugas 2: tombol hanya muncul jika ada minimal 1 tugas selesai */}
        {jumlahSelesai > 0 && (
          <button className="tombol tombol-merah tombol-kecil aksi" onClick={hapusSelesai}>
            Hapus Semua yang Selesai
          </button>
        )}
      </Kartu>

      <footer className="footer">
        © 2026 Adriel Reihansyah Setiawan · NPM 243040091
      </footer>
    </div>
  );
}

export default App;
```

Kartu.jsx



```jsx
pertemuan2 > src > Kartu.jsx > default
// src/Kartu.jsx
function Kartu({ judul, children }) {
  return (
    <section className="kartu">
      <h2>{judul}</h2>
      {children}
    </section>
  );
}

export default Kartu;
```

Penghitung.jsx

pertemuan2 > src > Penghitung.jsx > Penghitung

```jsx
// src/Penghitung.jsx (VERSI BENAR)
import { useState } from 'react';
import Kartu from './Kartu';

function Penghitung() {
  const [jumlah, setJumlah] = useState(0);

  const tambah = () => {
    setJumlah(jumlah + 1);
  };

  const ulangi = () => {
    setJumlah(0);
  };

  return (
    <Kartu judul="Penghitung Klik">
      <p>Tombol sudah diklik <span className="angka" key={jumlah}>{jumlah}</span> kali</p>

      {/* Conditional rendering: pesan hanya muncul jika jumlah >= 5 */}
      {jumlah >= 5 && <p>Wah, kamu rajin sekali mengklik!</p>}

      <div className="baris">
        <button className="tombol" onClick={tambah}>Klik Saya</button>
        <button className="tombol tombol-merah" onClick={ulangi}>Ulangi</button>
      </div>
    </Kartu>
  );
}

export default Penghitung;
```

FormTugas.jsx

pertemuan2 > src > FormTugas.jsx > default

```jsx
// src/FormTugas.jsx
import { useState } from 'react';

function FormTugas({ onTambah }) {
  const [teks, setTeks] = useState('');

  // Validasi: tidak boleh kosong dan maksimal 50 karakter
  const terlaluPanjang = teks.length > 50;
  const tidakValid = teks.trim() === '' || terlaluPanjang;

  const tanganiKirim = (event) => {
    event.preventDefault();      // cegah halaman reload
    if (tidakValid) return;      // hentikan jika tidak valid
    onTambah(teks.trim());       // "melapor" ke komponen induk
    setTeks('');                 // kosongkan input
  };

  return (
    <form onSubmit={tanganiKirim}>
      <div className="baris">
        <input
          className="input"
          type="text"
          placeholder="Contoh: Kerjakan laporan Modul 2"
          value={teks}
          onChange={(event) => setTeks(event.target.value)}
        />
        <button className="tombol" type="submit" disabled={tidakValid}>
          Tambah
        </button>
      </div>
      {terlaluPanjang && <p className="galat">Judul tugas maksimal 50 karakter.</p>}
    </form>
  );
}

export default FormTugas;
```

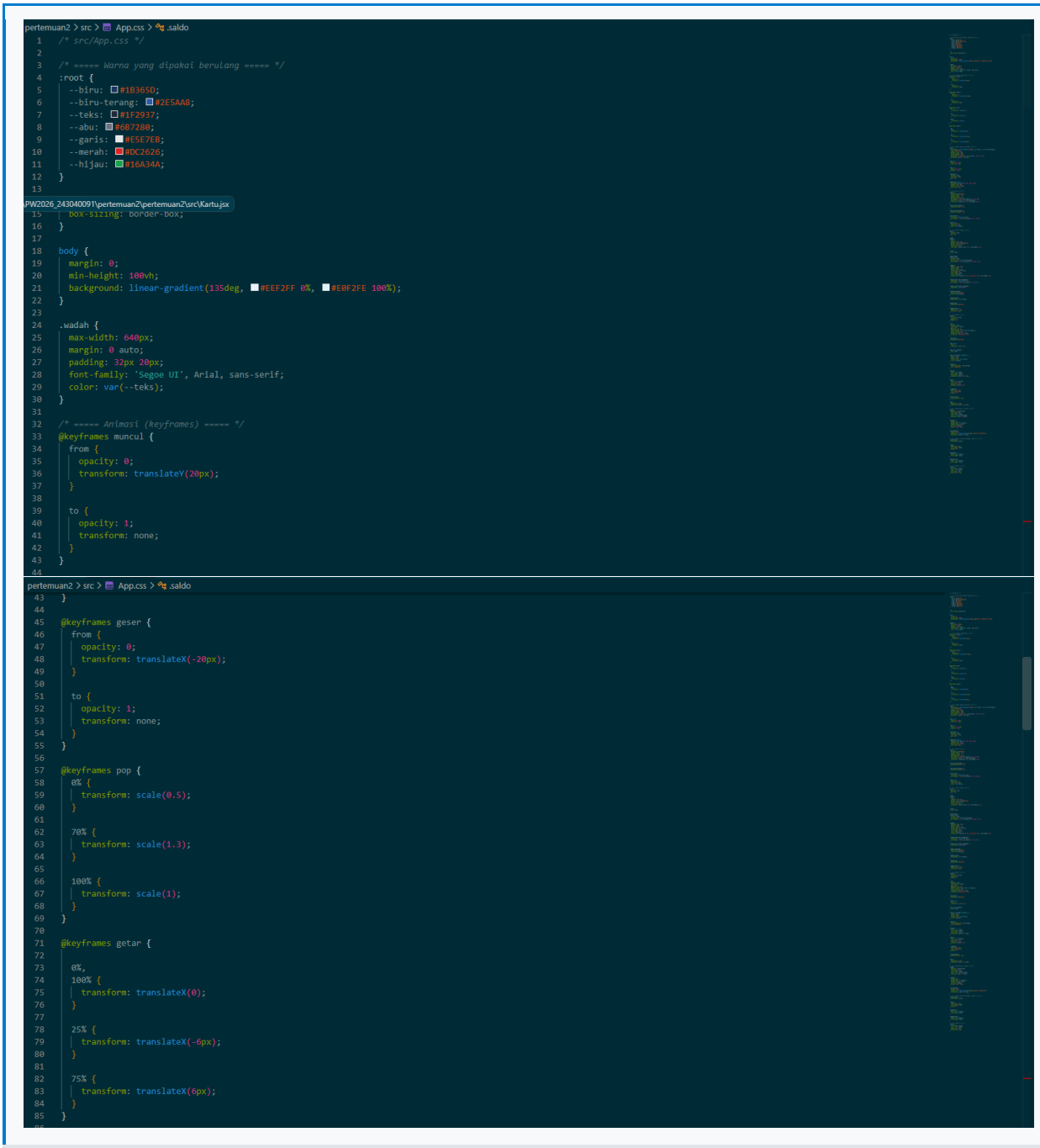
ItemTugas.jsx

pertemuan2 > src > ItemTugas.jsx > default

```jsx
// src/ItemTugas.jsx
function ItemTugas({ tugas, onUbahStatus, onHapus }) {
  return (
    <li className="item">
      <input
        type="checkbox"
        checked={tugas.selesai}
        onChange={() => onUbahStatus(tugas.id)}
      />
      {/* className dinamis: dicoret jika selesai */}
      <span className={tugas.selesai ? 'selesai' : ''}>{tugas.judul}</span>
      <button
        className="tombol tombol-merah tombol-kecil"
        onClick={() => onHapus(tugas.id)}
      >
        Hapus
      </button>
    </li>
  );
}

export default ItemTugas;
```

App.css



pertemuan2 > src > App.css > .saldo

```css
/* src/App.css */

/* ===== Warna yang dipakai berulang ===== */
:root {
  --biru: #1B365D;
  --biru-terang: #2E5AA8;
  --teks: #1F2937;
  --abu: #6B7280;
  --garis: #E5E7EB;
  --merah: #DC2626;
  --hijau: #16A34A;
}

\PW2026_243040091\pertemuan2\pertemuan2\src\Kartu.jsx
  box-sizing: border-box;
}

body {
  margin: 0;
  min-height: 100vh;
  background: linear-gradient(135deg, #EEF2FF 0%, #E0F2FE 100%);
}

.wadah {
  max-width: 640px;
  margin: 0 auto;
  padding: 32px 20px;
  font-family: 'Segoe UI', Arial, sans-serif;
  color: var(--teks);
}

/* ===== Animasi (keyframes) ===== */
@keyframes muncul {
  from {
    opacity: 0;
    transform: translateY(20px);
  }

  to {
    opacity: 1;
    transform: none;
  }
}
```

pertemuan2 > src > App.css > .saldo

```css
}

@keyframes geser {
  from {
    opacity: 0;
    transform: translateX(-20px);
  }

  to {
    opacity: 1;
    transform: none;
  }
}

@keyframes pop {
  0% {
    transform: scale(0.5);
  }

  70% {
    transform: scale(1.3);
  }

  100% {
    transform: scale(1);
  }
}

@keyframes getar {

  0%,
  100% {
    transform: translateX(0);
  }

  25% {
    transform: translateX(-6px);
  }

  75% {
    transform: translateX(6px);
  }
}
```

pertemuan2 > src > App.css > .saldo

```
  }

  /* ===== Header dengan identitas ===== */
  .hero {
    background: linear-gradient(135deg, var(--biru), var(--biru-terang));
    color: #FFFFFF;
    padding: 28px 24px;
    border-radius: 16px;
    margin-bottom: 24px;
    box-shadow: 0 10px 25px -5px rgba(27, 54, 93, 0.4);
    animation: muncul 0.6s ease;
  }

  .hero h1 {
    margin: 0 0 4px;
    font-size: 26px;
  }

  .hero p {
    margin: 0 0 16px;
    opacity: 0.85;
  }

  .identitas {
    display: flex;
    flex-wrap: wrap;
    gap: 8px;
  }

  .identitas span {
    background: rgba(255, 255, 255, 0.18);
    padding: 6px 12px;
    border-radius: 999px;
    font-size: 13px;
  }

  /* ===== Kartu ===== */
  .kartu {
    background: #FFFFFF;
    border-radius: 14px;
    padding: 20px;
    margin-bottom: 20px;
    box-shadow: 0 4px 10px rgba(0, 0, 0, 0.08);
    animation: muncul 0.5s ease backwards;
    transition: transform 0.2s, box-shadow 0.2s;
  }
```

```
}

.kartu:nth-of-type(2) {
  animation-delay: 0.1s;
}

.kartu:nth-of-type(3) {
  animation-delay: 0.2s;
}

.kartu:hover {
  transform: translateY(-3px);
  box-shadow: 0 12px 20px rgba(0, 0, 0, 0.12);
}

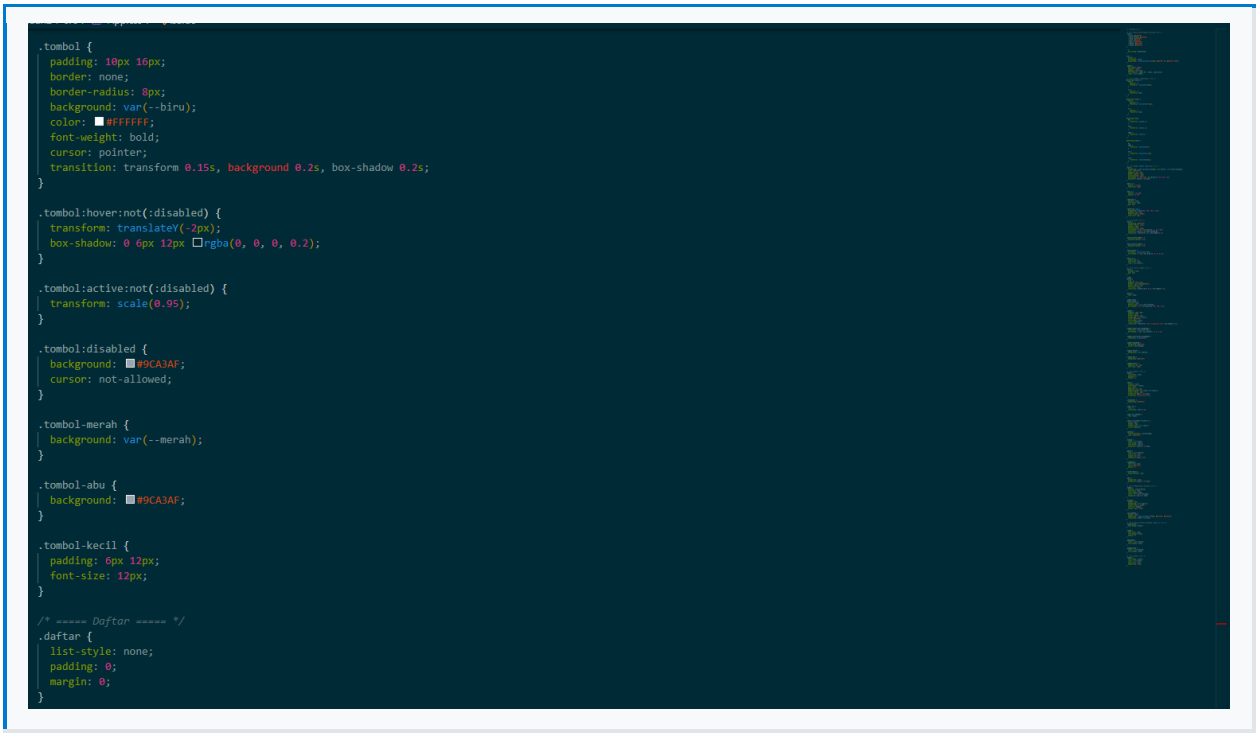
.kartu h2 {
  margin-top: 0;
  font-size: 18px;
  color: var(--biru);
}

/* ===== Form & tombol ===== */
.baris {
  display: flex;
  gap: 8px;
}

.input,
.pilih {
  flex: 1;
  padding: 10px 12px;
  border: 1px solid #D0D7DE;
  border-radius: 8px;
  font-size: 14px;
  transition: border-color 0.2s, box-shadow 0.2s;
}

.pilih {
  flex: none;
}

.input:focus,
.pilih:focus {
  outline: none;
  border-color: var(--biru-terang);
  box-shadow: 0 0 0 3px rgba(46, 90, 168, 0.2);
}
```



```css
.tombol {
  padding: 10px 16px;
  border: none;
  border-radius: 8px;
  background: var(--biru);
  color: #FFFFFF;
  font-weight: bold;
  cursor: pointer;
  transition: transform 0.15s, background 0.2s, box-shadow 0.2s;
}

.tombol:hover:not(:disabled) {
  transform: translateY(-2px);
  box-shadow: 0 6px 12px rgba(0, 0, 0, 0.2);
}

.tombol:active:not(:disabled) {
  transform: scale(0.95);
}

.tombol:disabled {
  background: #9CA3AF;
  cursor: not-allowed;
}

.tombol-merah {
  background: var(--merah);
}

.tombol-abu {
  background: #9CA3AF;
}

.tombol-kecil {
  padding: 6px 12px;
  font-size: 12px;
}

/* ===== Daftar ===== */
.daftar {
  list-style: none;
  padding: 0;
  margin: 0;
}
```

pertemuan2 > src > App.css > .saldo

```css
.item {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 12px 8px;
  border-bottom: 1px solid var(--garis);
  border-radius: 8px;
  animation: geser 0.3s ease;
  transition: background 0.2s;
}

.item:hover {
  background: #F8FAFC;
}

.item span {
  flex: 1;
  transition: color 0.3s;
}

.item span.nominal {
  flex: none;
}

.item input[type="checkbox"] {
  width: 18px;
  height: 18px;
  accent-color: var(--biru);
  cursor: pointer;
}

.selesai {
  text-decoration: line-through;
  color: #9CA3AF;
}

.kosong {
  color: var(--abu);
  text-align: center;
  font-style: italic;
  animation: muncul 0.4s ease;
}
```

pertemuan2 > src > App.css > .saldo

```css
}

.galat {
  color: var(--merah);
  font-size: 13px;
  margin: 6px 0 0;
  animation: getar 0.4s;
}

.ringkasan {
  font-size: 14px;
  color: #374151;
  margin: 0;
}

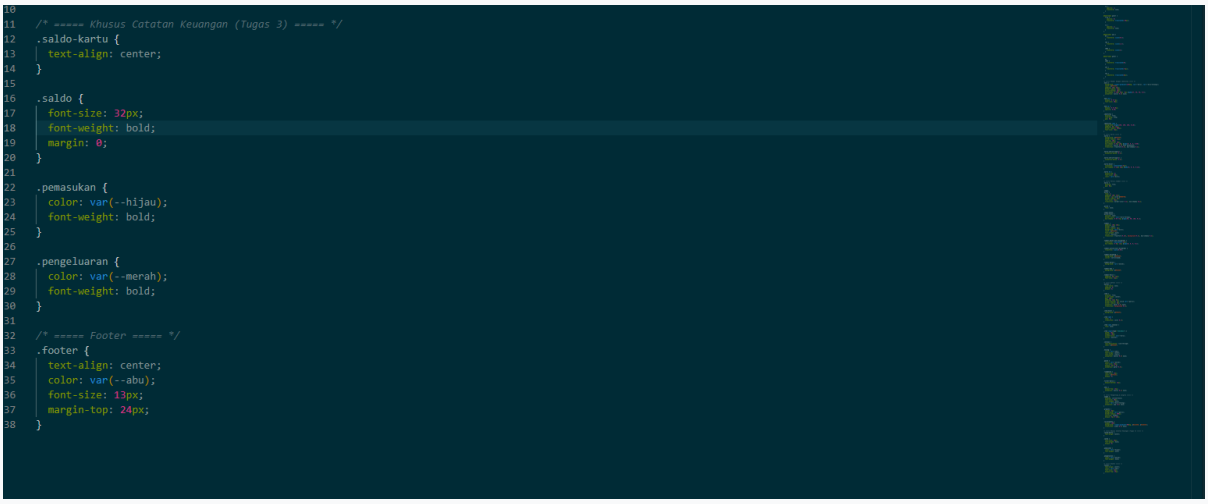
.filter-baris {
  margin-bottom: 12px;
}

.aksi {
  margin-top: 12px;
  animation: muncul 0.3s ease;
}

/* ===== Penghitung & progres ===== */
.angka {
  display: inline-block;
  font-size: 20px;
  font-weight: bold;
  color: var(--biru-terang);
  animation: pop 0.3s ease;
}

.progres {
  height: 8px;
  background: var(--garis);
  border-radius: 999px;
  overflow: hidden;
  margin: 8px 0 16px;
}

.isi-progres {
  height: 100%;
  background: linear-gradient(90deg, #22C55E, #16A34A);
  transition: width 0.5s ease;
}
```



```
/* ===== Khusus Catatan Keuangan (Tugas 3) ===== */
.saldo-kartu {
  text-align: center;
}

.saldo {
  font-size: 32px;
  font-weight: bold;
  margin: 0;
}

.pemasukan {
  color: var(--hijau);
  font-weight: bold;
}

.pengeluaran {
  color: var(--merah);
  font-weight: bold;
}

/* ===== Footer ===== */
.footer {
  text-align: center;
  color: var(--abu);
  font-size: 13px;
  margin-top: 24px;
}
```

Bagian 2. Penjelasan Kode

1. App.jsx

Ini komponen induk yang menyimpan data utama, yaitu daftarTugas, memakai useState. Ada juga state filter untuk memilih tampilan Semua, Belum Selesai, atau Selesai. Fungsi tambahTugas memakai spread (...), ubahStatus memakai .map(), dan hapusTugas serta hapusSelesai memakai .filter(). Semuanya membuat array baru dan tidak mengubah data lama, supaya React tahu ada perubahan dan layar diperbarui. Jumlah tugas selesai dan persen progress dihitung langsung dari state, jadi tidak perlu dibuat state baru.

App juga memakai tiga useEffect: satu untuk menampilkan pesan di Console saat aplikasi pertama dibuka, satu untuk mengubah judul tab sesuai jumlah tugas yang belum selesai, dan satu untuk menyimpan data ke localStorage setiap daftar berubah.

Kartu.jsx

Komponen bingkai kartu yang bisa dipakai berulang. Judulnya dikirim lewat props, sedangkan isinya memakai props children, yaitu apa pun yang ditaruh di antara tag Kartu.

2. Penghitung.jsx

Komponen latihan untuk memahami useState. Angka klik disimpan di state, dan setiap tombol Klik Saya ditekan angkanya bertambah. Tombol Ulangi mengembalikan angka ke 0. Pesan “Wah, kamu rajin sekali mengklik!” hanya muncul jika klik sudah 5 kali atau lebih (conditional rendering dengan &&).

3. FormTugas.jsx

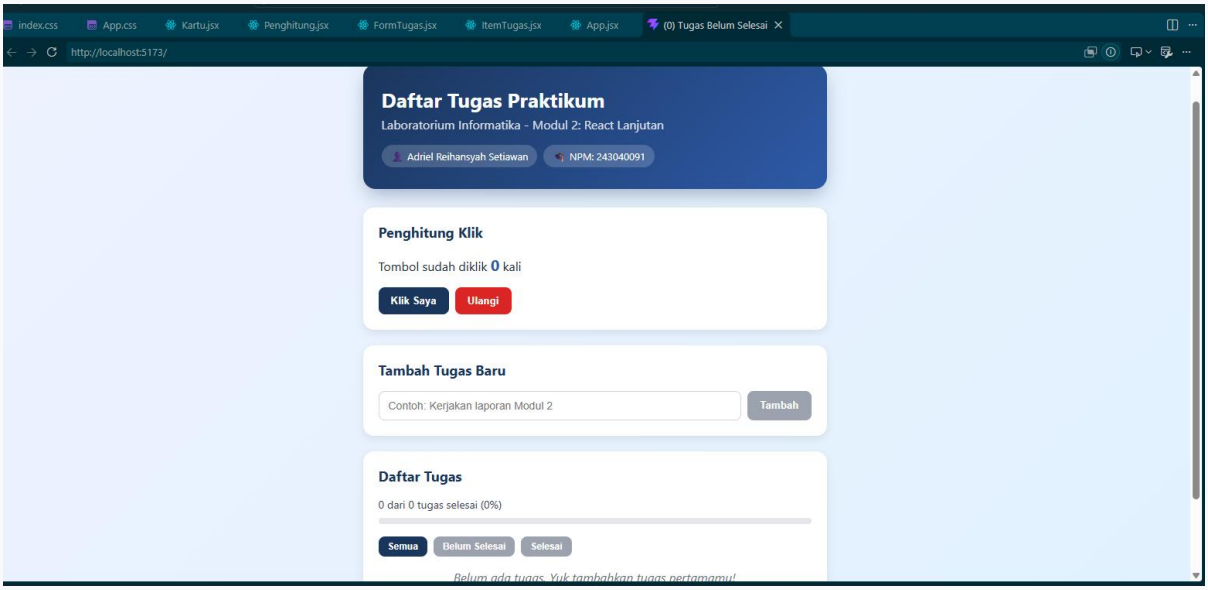
Formulir untuk menambah tugas. Isi input disimpan di state teks, dan input dihubungkan dengan value dan onChange (disebut controlled form). Tombol Tambah akan nonaktif jika input kosong atau lebih dari 50 karakter. Saat form dikirim,

event.preventDefault() dipakai agar halaman tidak reload, lalu komponen ini memanggil fungsi onTambah dari App untuk menambahkan tugas.

4. ItemTugas.jsx
Komponen untuk menampilkan satu tugas, berisi checkbox, judul, dan tombol Hapus. Jika tugas selesai, judulnya dicoret lewat className dinamis. Tombol Hapus memakai arrow function supaya fungsi hanya berjalan saat diklik, bukan saat halaman dimuat.
Fitur Tugas 1 (Filter)
Dibuat state filter dengan tiga pilihan. Daftar disaring dengan .filter() sebelum ditampilkan dengan .map(). Tombol filter yang aktif berwarna biru dan yang tidak aktif berwarna abu-abu.
Fitur Tugas 2 (Hapus Semua yang Selesai)
Tombol ini hanya muncul jika ada minimal satu tugas yang selesai (memakai &&). Saat diklik, semua tugas selesai dihapus dengan .filter().

5. App.css
Berisi semua gaya tampilan, termasuk animasi sederhana seperti kartu muncul dan item meluncur, yang dibuat dengan CSS saja.

Bagian 3. Screenshot Hasil Antarmuka



Bagian 4. Penjelasan Hasil Antarmuka

1. Tampilan awal: Halaman menampilkan header berisi nama dan NPM, lalu tiga kartu (Penghitung, Tambah Tugas, dan Daftar Tugas). Daftar berisi dua tugas contoh beserta progress bar.
2. Penghitung: Angka di layar ikut bertambah setiap tombol diklik karena disimpan di state. Setelah 5 klik, muncul pesan tambahan.
3. Tambah tugas: Setelah mengetik dan menekan Enter, tugas baru muncul di daftar dan input kosong lagi. Halaman tidak reload karena ada preventDefault.
4. Lebih dari 50 karakter: Muncul pesan galat merah dan tombol Tambah nonaktif. Ini bisa terjadi karena isi input selalu sama dengan state sehingga bisa langsung dicek.
5. Centang tugas: Tugas yang dicentang jadi tercoret, dan ringkasan serta progress bar ikut berubah.
6. Filter Belum Selesai: Hanya tugas yang belum dicentang yang tampil, dan tombol filter ini berwarna biru.
7. Filter Selesai: Hanya tugas yang sudah dicentang yang tampil.
8. Hapus Semua yang Selesai: Tombol muncul saat ada tugas selesai. Setelah diklik, tugas selesai hilang dan tombolnya ikut hilang.
9. Hapus semua tugas: Muncul pesan “Belum ada tugas. Yuk tambahkan tugas pertamamu!” karena daftar kosong.
10. Judul tab: Judul tab berubah sesuai jumlah tugas yang belum selesai, misalnya “(2) Tugas Belum Selesai”. Ini hasil dari useEffect.
11. Setelah refresh: Data tugas tetap ada karena sudah disimpan di localStorage, tidak hilang seperti sebelum memakai useEffect.
12. Local Storage: Di tab Application terlihat kunci daftar-tugas yang berisi data tugas dalam bentuk teks JSON.
13. Console: Pesan “Aplikasi Daftar Tugas siap digunakan!” muncul dua kali. Ini normal karena React.StrictMode sengaja menjalankan efek dua kali saat development.